

On Optimizing The Longest Common Subsequence Problem by Loop Unrolling Along Wavefronts

Johann Steinbrecher and Weijia Shang

Santa Clara University

500 El Camino Real

Santa Clara, Ca 95053

j1steinbrecher@scu.edu, wshang@scu.edu

Abstract—Loop unrolling is a loop transformation where a few loop iterations are grouped as a *super iteration* for exploring more independent instructions and to decrease the total loop overhead. This paper characterizes loop unrolling by the *unrolling factor*, the number of iterations in a super iteration and the *unrolling direction*, the choice of iterations to be grouped to form the super iteration. We use loop unrolling for maximizing instruction-level parallelism in the longest common subsequence problem. To increase the number of independent instructions in the super iteration, we use a linear schedule to group iterations on the same *wavefront*, a hyperplane in the loop iteration space. Then, the loop is unrolled along the wavefront which guarantees all iterations in the same super iteration are independent. The selection of the optimal unrolling factor is based on the assumption that if all the pipelines are saturated, the performance should not be bad. Two necessary conditions and a sufficient condition for optimality are presented and used to find the optimal unrolling factor. The total execution time is expressed as a function of algorithm parameters, architecture parameters and the unrolling factor. A benchmark of the technique scores a 1.475 speed-up over traditional methods.

I. INTRODUCTION

Loop unrolling is a loop transformation where a few loop iterations are grouped as a super iteration so that more independent instructions are available to feed multiple pipelines in a system. In this paper we address how to unroll a loop in an optimal way and how to find the optimal size of the super iteration.

Loop unrolling exposes many characteristics of a loop which can be exploited to improve performance. On one hand, loop unrolling can cut the number of instructions, especially the load/save instructions because a loaded variable can be reused in the super iteration. It also cuts the branch instructions and the increment instructions because the number of iterations are smaller after the unrolling. However, unrolling a loop can also have negative impact on the performance. The execution of the unrolled loop needs more registers to hold intermediate values. If the register usage of a highly unrolled loop exceeds the number of available registers, register spilling will occur which could delay the completion of the execution significantly. Hence, one challenge in loop unrolling is to find the smallest or *optimal* unrolling factor such that all pipelines in the system are utilized. The optimal unrolling factor depends on the dependence structure of the super iteration. Intuitively, if more instructions are independent of each other in the super iteration, the smaller the optimal unrolling factor will

be. Hence, another challenge of loop unrolling is to group independent iterations to form the super iteration, i.e., find the *optimal unrolling direction*.

How to find the optimal unrolling factor is also addressed in [12] where a cost function of a perfectly nested loop is formulated in terms of the unrolling factor u . The cost function considers register spills and the *critical path length* that are affected by the unrolling factor. Based on the cost function it calculates an unrolling factor that optimizes the execution time of the loop. While Sarkar emphasizes in [12] on finding a cost model to identify optimal unrolling factors for certain unrolling directions, he does not investigate on finding optimal unrolling directions. If dependence exists along the unrolling direction, performance can be lost significantly. Later in this paper, it is shown that if the unrolling direction is not optimal, pipelines in the system cannot be saturated no matter how large the unrolling factor. In this paper, the unrolling direction is taken into consideration. Super iterations are formed to contain independent instructions. This improves the performance especially for the case where dependence exists along the inner loop.

In this paper, *wavefronts* or *linear schedules* are used to identify independent iterations in the loop nest. A linear schedule can partition the iteration space of the loop to be unrolled into parallel hyperplanes or wavefronts where all iterations on the same wavefront are independent. Then the loop is unrolled along a wavefront which guarantees that all iterations in the same wavefront are independent. Having found the unrolling direction, the next step is to find the optimal unrolling factor: the number of times a loop nest needs to be unrolled in order to fully utilize the pipelines of the hardware. If the pipelines are saturated, the execution time is likely optimized. A sufficient condition and two necessary conditions are presented and used to find the optimal unrolling factor. Then the total execution time is expressed as a function of a set of algorithm parameters and architecture parameters.

Section III.B of paper [3] presents a skewed loop nest of the longest common subsequence (*LCS*) problem that eliminates loop carried dependencies along the inner loop. However, the emphasis of this paper is on thread-level parallelism not on exploiting instruction-level parallelism (*ILP*). Parallelyzing loops was studied early by [8] who used hyperplanes in the code which helps to execute a loop nest in a multi processor architecture. Manjikian investigates in [10] how to schedule

wavefronts to scalable shared-memory multiprocessors to exploit parallelism. In order to guarantee ILP [9] and [16] present loop transformation techniques that are supposed to decrease the data dependence of instructions in one super iteration. The result is a shorter critical path length and a higher degree of parallelism. Paper [1] describes a loop transformation that generates parameterized parallel code out of a nested loop. To evaluate an expected execution time, equations have been established to estimate their cycles of execution. Bondhugula presents in [4] an automatic polyhedral source-to-source compilation framework that parallelizes and improves the locality of regular dependence programs by generating multithreaded code using OpenMP pragmas. Paper [17] presents a model that estimates the total machine cycle time taking into account cache misses, software pipelining, register pressure and loop overhead. Paper [12] introduces a cost model considering ILP and I-cache for choosing optimal unrolling factors. How often a loop should be unrolled in order to optimize its performance and how to automatically select this unrolling factor is subject of several papers. [15] demonstrates how simple machine learning algorithms can be used to find good or optimal unrolling factors. Paper [13] focuses on how to select an unrolling factor that is optimal for the given register file, i.e. a superword register file. This approach finds that the bottleneck of unrolling loops is the size of the register file. However, it disregards the number of functional units as a parameter in finding an optimal unrolling factor.

The rest of the paper is organized as follows. Section II presents architecture and algorithm models and the concepts of linear schedules and wavefronts. Section III describes the basic ideas. Section IV describes necessary and sufficient conditions for the optimal unrolling factor as well as an equation for the total execution time. Section V applies the technique and equations on the LCS problem. In section VI the new technique is benchmarked. Section VII concludes this paper.

II. ARCHITECTURAL AND ALGORITHM MODEL

In this section we present an overview on our algorithm model, architectural model and linear schedule. This paper applies loop unrolling along wavefronts to the longest common subsequence problem. However, the basic ideas and concepts indicated apply generally to perfectly nested loops with uniform dependencies. The longest common subsequence algorithm takes as inputs two sequences of characters: $x[1...M]$ and $y[1...N]$. The program then attempts to find the longest subsequence that is shared by these two strings, and returns the length of that subsequence. For example, for the input strings 'ABCBDAB' and 'BDCABA', the program will output four, because the longest common subsequence is 'BCBA', which has a length of four. The dynamic programming version of the longest common subsequence problem can be found in ([6], pp. 353) and presented in figure 1. The LCS problem is well-known for its application in bio-engineering for DNA matching and as the kernel of the UNIX *diff* program.

A. Algorithm model

We only consider algorithms with regular dependencies [11], i.e. algorithms containing array references such that

```
for (i=0; i <= M; i++) {
  for (j=0; j <= N; j++) {
    if (i == 0 || j == 0)
      c[i][j] = 0;
    else if (x[i] == y[j])
      c[i][j] = c[i-1][j-1] + 1;
    else
      c[i][j] = max(c[i][j-1], c[i-1][j]);
  }
}
```

Fig. 1. The dynamic programming solution to the LCS problem [6].

the dependencies remain constant from one iteration to the next. Such algorithms may be described by two parameters, the dependence matrix D and the iteration space J . For the LCS problem, the dependence matrix D is:

$$D = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$$

The iteration space is the set of all iteration indexing vectors and shown in figure 2. The process of establishing the dependence matrix and the iteration space is beyond the scope of this paper and can be found in [11]. Intuitively, each point in the iteration space corresponds to an iteration in the loop. An arrow between two points indicates a dependence between the two iterations.

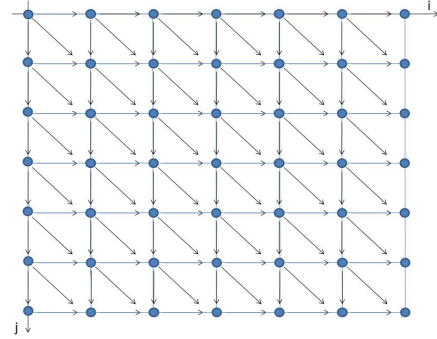


Fig. 2. Iteration space for the longest common subsequence problem.

The assembly code of the loop body for the LCS problem is presented in figure 3. The notation I_i in figure 3 corresponds to the i th instruction in the assembly code. The definitions of these assembly instructions are presented in figure 4. Depending on if the two current characters from the two strings match, different operations are done in each iteration. The case when most instructions are executed is assumed in the assembly code. Also, the last two instructions: the increment and branch instructions are not shown in the code. This simpler and incomplete assembly code will be used to illustrate basic ideas and concepts we wish to present later to avoid the complications caused by the complete assembly code of the LCS problem. This will not lose generality and correctness of our method because it is based on the algorithm's parameters. The incomplete and complete assembly codes of the LCS problem have different algorithm parameters. Later

at benchmarking the method, the complete assembly code of the LCS problem is used.

```
I1: load x[i], R1
I2: load y[j], R2
I3: cmp R1, R2
I4: load c[i][j-1], R3
I5: load c[i-1][j], R4
I6: max R5, R3, R4
I7: store c[i][j], R5
```

Fig. 3. The assembly code of one iteration of the longest common subsequence problem.

Instruction	Definition
load x[i], R1	load the memory value x[i] into register R1
store c[i][j], R3	store the value of register R3 into memory location c[i][j]
cmp R1, R2	compare registers R1 and R2
max R3, R1, R2	save the max(R1,R2) into register R3

Fig. 4. Definition of the assembly instructions used in this paper.

Dependencies exist between assembly instructions. A graph is used to describe the dependence between instructions where a node corresponds to an assembly instruction and an arc indicates a direct dependence between the two instructions. The data dependence graph (DDG) for the simplified assembly code of the LCS problem is shown in figure 5. The DDG is divided into $k \geq 2$ subsets or dependence stages (DS) $DS_1, \dots, DS_k$. DS_i consists of the i th instructions of all directed paths in the DDG. The instruction count in DS_i is denoted as Ic_i . For example, as shown in figure 5, DS_1 contains $I1$ and $I2$. Therefore the instruction count of dependence stage 1 Ic_1 equals 2. The longest directed path is called *critical path* and its length is denoted as lcp , which equals the number of dependence stages. In the example in figure 5, $lcp = 5$.

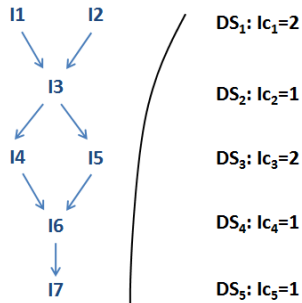


Fig. 5. DDG of the assembly instructions in figure 3.

Therefore, at the assembly level, the algorithm is modeled by the following parameters: DDG, lcp and the instruction counts Ic_i , $i = 1, \dots, lcp$. In summary, algorithms are modeled by the following set of parameters D, J, DDG, lcp, Ic_i , $i = 1, \dots, lcp$.

B. Linear schedule and wavefronts

The linear schedule $f(\Pi)$ is a linear or affine function that maps multiple dimensional iteration vectors in the iteration space onto a set of execution times represented by integers. Multiple independent iterations are assigned the same execution time for parallel processing. How to identify linear schedules that respect the data dependencies and minimize the total execution time is discussed in [11]. For the dependence structure of the LCS problem in figure 2, one feasible linear schedule is

$$f([i \ j]^t) = [1 \ 1][i \ j]^t = i + j$$

The vector $[1 \ 1]$ specifying the linear schedule is called *linear schedule vector* and is denoted as Π . The iteration space is partitioned by the linear schedule into a series of hyperplanes $[1 \ 1][i \ j]^t = \text{constant}$. These hyperplanes (diagonal lines in this example) are called *wavefronts* that are perpendicular to Π (see [11]). Such wavefronts may be generated using CLoG [2], a loop transformation and code generation tool based on the polyhedral model. The wavefronts $[1 \ 1][i \ j]^t = 0, \dots, 13$ are shown in figure 6. One observation is that all iterations in the same wavefront are independent.

C. Architecture model

A computer system is modeled by the parameters $(p, p_{stage}, n_{regfile}, n_{str})$ where p is the number of pipelines in the system, p_{stage} is the number of stages in each pipeline, $n_{regfile}$ is the number of registers, and n_{str} is the number of stalls to resolve a pipeline hazard. It is assumed it takes the same number of stalls to resolve all kinds of hazards (control or data hazard). Other architectural features such as ALU arithmetic as well as memory and cache access latencies have been abstracted away to simplify our calculations. They will be considered in future research. Clearly, the total execution time of an algorithm is also a function of the architecture parameters.

III. BASIC IDEA

The advantages of unrolling the LCS along wavefronts are illustrated by examples in this section. In section III-A, the loop is unrolled along the inner loop, the traditional way, and the pipelines are scheduled according to the corresponding DDG. In section III-B, the loop is unrolled along the wavefront and the pipelines are scheduled according to the corresponding DDG. Section III-C compares the two unrolling transformations and discusses the advantages and disadvantages of loop unrolling along the wavefronts. Again, because all iterations are independent when unrolling along wavefronts, the critical path in the corresponding DDG is much shorter and the total execution time is much shorter.

A. Unrolling along the inner loop

The most common method to apply loop unrolling to this problem would be to unroll along the inner loop. The code for unrolling along the inner loop is given in figure 7, with an unrolling factor of $u = 2$:

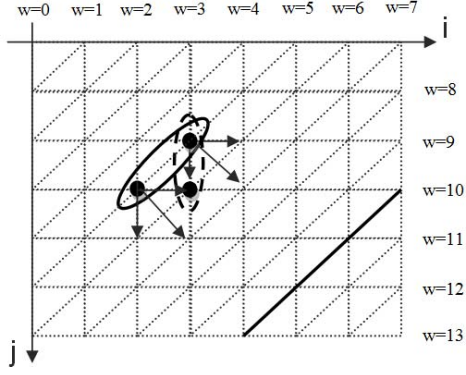


Fig. 6. Wavefronts and super iterations in the iteration space of the LCS problem. The diagonal lines, the wavefronts, are defined by the linear schedule vector $[1 \ 1]$ and indexed by $w = i + j$. The dashed oval outlines one super iteration of unrolling along the inner loop and corresponds to the code of figure 7. The solid oval outlines a super iteration which unrolls the loop along the wavefront (code in figure 11). At wavefront 10 there are four iterations in the wavefront and therefore, $wave[10] = 4$. There are in total 14 wavefronts in this iteration space ($P = 14$).

```

for (i=0; i<=M; i++) {
  for (j=0; j<N; j+=2) {
    if (i == 0 || j == 0)
      c[i][j] = 0;
    else if (x[i] == y[j])
      c[i][j] = c[i-1][j-1] + 1;
    else
      c[i][j] = max(c[i][j-1], c[i-1][j]);

    if (i == 0 || j+1 == 0)
      c[i][j+1] = 0;
    else if (x[i] == y[j+1])
      c[i][j+1] = c[i-1][j] + 1;
    else
      c[i][j+1] = max(c[i][j], c[i-1][j+1]);
  }
}

```

Fig. 7. High level language code of the LCS problem unrolled along the inner loop with an unrolling factor of $u = 2$.

The corresponding assembly code is shown in figure 8 and its DDG is presented in figure 9. The critical path length lcp is found to be 6.

Note that $c[i][j]$ is used in both iterations of unrolled code, one write and one read. This results in a read-after-write dependence between the two iterations. That is why the length of the critical path is increased from five to six. One super iteration of this unrolling is indicated by the dashed oval in figure 6. It shows dependent iterations are grouped together. In general, the length of the critical path is a linear function of the unrolling factor if chain dependence exists among iterations in one super iteration.

The instructions in figure 9 are scheduled for execution optimality in figure 10. The architecture parameters are assumed as follows: $p = 3$, $n_{str} = 2$. The total execution time for one super iteration is $18 + p_{stage}$ clock cycles.

This unrolling made some improvements to the original

```

I1: load x[i], R1
I2: load y[j], R2
I3: cmp R1, R2
I4: load c[i][j-1], R3
I5: load c[i-1][j], R4
I6: max R5, R3, R4
I7: store c[i][j], R5
I8: load y[j+1], R6
I9: cmp R1, R6
I10: load c[i-1][j+1], R7
I11: max R7, R7, R5
I12: store c[i][j+1], R7

```

Fig. 8. The assembly code of one super iteration of the loop in figure 7.

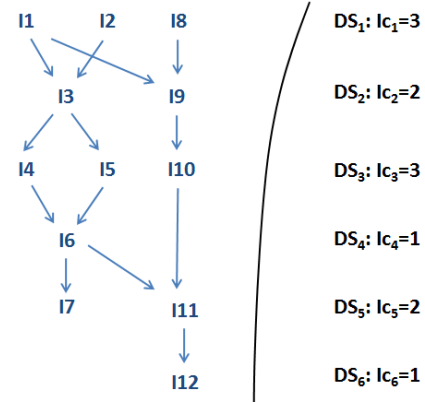


Fig. 9. Dependence graph for the code in figure 8 ($u = 2$).

code in figure 1. It eliminated some loop overheads, so that j will only be incremented half as many times, and branches will be executed half as many times. It also eliminated some repetitive instructions. E.g., the element $c[i-1][j]$ is used by both iterations of the original code, so $c[i-1][j]$ only needs to be loaded once.

B. Unrolling along wavefronts

As mentioned earlier, it is best to group independent iterations to form a super iteration and all iterations on the same wavefronts are guaranteed to be independent. As discussed in section II-B, one feasible linear schedule vector is $[1 \ 1]$ and the wavefronts $w = i + j$ defined by this vector are shown by the diagonal lines in figure 6.

The code describing the unrolling along wavefronts with $u = 2$ is presented in figure 11. The original loop has been transformed such that the outer and inner loops no longer use i and j as index, instead now it uses the index of wavefronts and index of points along each wavefront. Let P be the number of wavefronts and let all wavefronts be defined as $w = i + j$, $w = 0, \dots, P-1$. Also, let $wave[w]$ contain the number of points on wavefront $i + j = w$. The variable w iterates from 0 to $P-1$, while the index of the inner loop v now iterates from 0 to $wave[w] - 1$. During each iteration, the inner loop is unrolled to $[i \ j]$ and $[i-1 \ j+1]$. The corresponding assembly code and its DDG are shown in figure 12 and figure 13, respectively.

	$\xleftarrow{n_{str}}$																	
P1	I1	x	x	I3	x	x	I4	x	x	I6	x	x	I7	x	x	I12	x	x
P2	I2	x	x	I9	x	x	I5	x	x	x	x	x	I11	x	x	x	x	x
P3	I8	x	x	x	x	x	I10	x	x	x	x	x	x	x	x	x	x	x

Fig. 10. Filling the pipelines for the dependence graph in figure 9. The architecture's pipelines are supposed to be $p = 3$ and $n_{str} = 2$.

```

for (w=0; w<P; w++) {
  for (v=0; v<wave[w]; v+=2) {
    i = min(M,w) - v;
    j = max(0,w-M) + v;

    if ((i == 0) || (j == 0))
      c[i][j] = 0;
    else if (x[i] == y[j])
      c[i][j] = c[i-1][j-1]+1;
    else
      c[i][j] = max(c[i][j-1], c[i-1][j]);

    k = i-1;
    l = j+1;
    if (k>=0 && l<=N) {
      if ((k == 0) || (l == 0))
        c[k][l] = 0;
      else if (x[k] == y[l])
        c[k][l] = c[k-1][l-1]+1;
      else
        c[k][l] = max(c[k][l-1], c[k-1][l]);
    }
  }
}

```

Fig. 11. High level language code of the LCS problem unrolled along the wavefronts with an unrolling factor of $u = 2$.

```

I1: load x[i], R1
I2: load y[j], R2
I3: cmp R1, R2
I4: load c[i][j-1], R3
I5: load c[i-1][j], R4
I6: max R5, R3, R4
I7: store c[i][j], R5
I8: load x[k], R6
I9: load y[l], R7
I10: cmp R6, R7
I11: load c[k][l-1], R8
I12: load c[k-1][l], R9
I13: max R10, R8, R9
I14: store c[k][l], R10

```

Fig. 12. The assembly code of one super iteration of the loop in figure 11.

To illustrate the new method let us focus on unrolling along wavefront index $w = 5$, the one with the solid oval in figure 6. As there are six points on this wavefront ($wave[5] = 6$), v iterates from 0 to 5. During the first iteration, v is 0, hence $i = w = 5$ and $j = v = 0$; then $k = i - 1 = 4$, and $l = j + 1 = 1$. Hence, the first iteration builds a super iteration with a pair (5,0) and (4,1), which are the first two points on this wavefront. The next iteration will form a super iteration out of (3,2) and (2,3) as shown by the solid oval in figure 6.

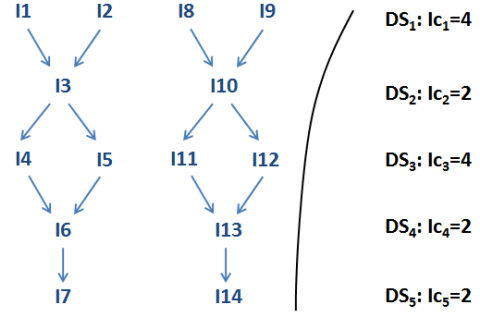


Fig. 13. Dependence graph of the assembly code in figure 12.

To demonstrate the utilization of the pipelines and the execution time for one super iteration, figure 14 shows the layout for the unrolled code in figure 12 with the same architectural parameters as in section III-A $p = 3$ and $n_{str} = 2$. The execution time is $16 + p_{stage}$ clock cycles.

C. Discussion

As seen from figure 10, the unrolling along inner loop requires $18 + p_{stage}$ cycles to complete, while the new unrolling technique (figure 14) requires only $16 + p_{stage}$ cycles. The key reason for this improvement is that the length of the critical path does not increase with the unrolling factor in the latter case and the completion time can not be shorter than the critical path length. Therefore, it is possible to cut more and more loop overhead with increasing unrolling factors without increasing the critical path length. In contradiction by unrolling along the inner loop the critical path length increases with the unrolling factor. This causes the execution time to increase independent of any architectural parameters. This is one crucial advantage. Loop unrolling along wavefronts can saturate massively parallel systems without increasing the critical path length of its super iterations. Therefore, it can take the full advantage out of the decrease of the loop overhead. To saturate the available pipelines as good as possible, the unrolling factor for unrolling along the inner loop has to be much larger than for our new method. Unrolling with a larger

	$\xleftarrow{n_{str}}$															
P1	I1	I9	x	I3	I10	x	I4	I11	x	I6	I13	x	I7	I14	x	x
P2	I2	x	x	x	x	x	I5	I12	x	x	x	x	x	x	x	x
P3	I8	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x

Fig. 14. Filling the hardware's pipelines for the dependence graph in figure 13. The architecture's pipelines are supposed to be $p = 3$ and $n_{str} = 2$.

u needs much more registers and may cause register spilling. One advantage of unrolling along the inner loop is that more load instructions may be eliminated due to the dependence chain. One observation is that $u = 2$ may not be the optimal unrolling factor because many cycles are unused as indicated by figure 14.

IV. OPTIMAL UNROLLING FACTOR AND THE TOTAL EXECUTION TIME

This section discusses how to find the optimal unrolling factor for given algorithm and architecture parameters. Two necessary conditions and one sufficient condition for the optimal unrolling factor are presented. Then, the total execution time is expressed as a function of the unrolling factor and other architecture and algorithm parameters. The impact of register spilling is not considered. In other words an infinite number of registers is assumed.

A. Parameters that are functions of the unrolling factor u

We only consider the assignment of registers to array subscript elements in our analysis. It is assumed that the number of used registers increases linearly with the unrolling factor u . This register usage requirement only holds true for small unrolling factors, so it suffices for our model. Therefore, the number of registers n_{reg} equals the initial number of registers n_{reg0} plus the additional number of registers caused by unrolling $n_{reg1} \cdot u$.

$$n_{reg} = n_{reg0} + n_{reg1} \cdot u$$

The total number of instructions n_I in a DDG and the number of instructions at each dependence stage Ic_i are also modeled as affine functions of u , i.e., the number of instructions in $Ic_i(u)$ equals the initial number of instructions Ic_{i0} plus the additional number of instructions caused by unrolling $Ic_{iu} \cdot u$.

$$Ic_i(u) = Ic_{i0} + Ic_{i1} \cdot u$$

$$\text{Hence, } n_I = \sum_{i=1}^{lcp} (Ic_i) = \sum_{i=1}^{lcp} (Ic_{i0}) + u \cdot \sum_{i=1}^{lcp} (Ic_{i1})$$

B. Time to execute the critical path in a super iteration (t_{cp})

In the critical path of a super iteration, there are lcp instructions that have to be executed in proper sequence. Let these instructions be $I1, I2, I3, \dots, Ilcp$. As illustrated by figure 15 the first instruction $I1$ will take p_{stage} cycles to go through the pipeline. Thereafter, each of the following instructions will take $(1 + n_{str})$ cycles to complete their executions through the pipeline. Let t_{cp} be the execution time of all instructions in the critical path. Then

$$t_{cp} = p_{stage} + lcp \cdot (n_{str} + 1)$$

Due to the dependencies in the critical path the time t_{cp} is the lower bound for the execution time of a super iteration.

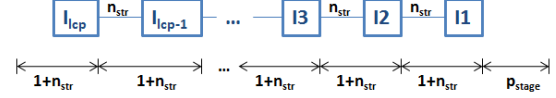


Fig. 15. Sequential execution of instructions in a pipeline.

C. Necessary and sufficient conditions for the optimal unrolling factor u_{opt}

The *optimal unrolling factor* is defined as the minimum unrolling factor u_{opt} that saturates the pipelines without any stalls. As the instruction counts $Ic_i, i = 1, \dots, lcp$ and hence the total number of instructions n_I increase linearly with the unrolling factor the pipelines should be filled with more instructions and the number of stalls should be decreased. Such a defined optimal unrolling factor may not exist if the critical path length increases faster than the number of independent instructions as indicated later by the LCS unrolled along the inner loop. In order to find the optimal unrolling factor that saturates the pipelines, two necessary conditions are established in the following. In the first case, it is assumed that instructions at stage i will not be executed until all the instructions in stage $i - 1$ are executed. Consider the execution of instructions in stage i , because it takes $n_{str} + 1$ cycles to resolve the data hazards, $p \cdot (n_{str} + 1)$ instructions are needed to fill the p pipelines. Therefore, the first necessary condition for the optimal unrolling factor u_{opt} is as follows.

Necessary condition 1:

Consider the case where the instructions at stage i will not be executed until all the instructions in stage $i - 1$ are executed. If all the p pipelines are saturated then the following is true:

$$\min\{Ic_1, Ic_2, \dots, Ic_{lcp}\} \geq p \cdot (n_{str} + 1)$$

If instructions at stage i are allowed to be executed before all the instructions in stage $i - 1$ are executed as long as the dependencies are respected, the necessary condition will be less restrictive.

Necessary condition 2:

Consider the set of all instructions IS . If there exists a partition $\{IS_1, \dots, IS_{lcp}\}$ such that:

$$IS_i \cap IS_j = \emptyset, i \neq j$$

$$\{IS_1 \cup IS_2 \cup \dots \cup IS_{lcp}\} = IS$$

and all instructions in $IS_i, i = 1, \dots, lcp$ are independent, then if the p pipelines are saturated, the following is true:

$$\min\{|IS_1|, |IS_2|, \dots, |IS_{lcp}|\} \geq p \cdot (n_{str} + 1)$$

For example figure 16 shows a dependence graph with two dependence stages after loop unrolling. Let's assume $p = 2$ and $n_{str} = 2$. The first necessary condition is not satisfied since $IC_2 < (p \cdot n_{str} + 1)$, i.e. $4 < 6$. However, if $IS_1 = \{I1, I2, \dots, I6\}$ and $IS_2 = \{I7, I8, I9, I10, I11, I12\}$, the second necessary condition is satisfied since

$$\begin{aligned} \min\{|IS_1|, |IS_2|\} &\geq p \cdot (n_{str} + 1) \\ \min\{6, 6\} &\geq 3 \cdot (2 + 1) \\ 6 &\geq 6 \end{aligned}$$

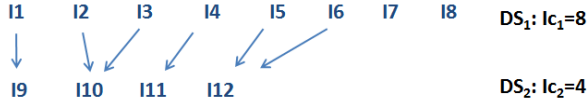


Fig. 16. A dependence graph to illustrate the second necessary condition.

A sufficient condition for the optimal unrolling factor can be found by dividing the dependence graph into classes where each class represents a dependence subgraph that has no dependence links into other classes. For instance consider the dependence graph in figure 13, instructions $I1, I2, \dots, I7$ form one class and instructions $I8, I9, \dots, I14$ form another class. As each dependence stage in each class contains at least one instruction, the pipelines will be saturated if there are at least $(n_{str} + 1) \cdot p$ classes with the same number of dependence stages. This guarantees there are at least $(n_{str} + 1) \cdot p$ independent instructions at each dependence stage and the p pipelines will be saturated. This condition is formally presented as follows.

Sufficient condition:

If there are at least $(n_{str} + 1) \cdot p$ classes in the DDG with the same number of dependence stages, then all pipelines will be saturated.

D. Formula for calculating the total execution time t_{total}

In this subsection, the formula for the execution time of one super iteration is derived first. Then, the total execution time of the whole loop is obtained.

Let the execution time of one super iteration be denoted as $t_{superiter}$. One lower bound is t_{cp} , the execution time of all instructions on the critical path. Let $n_I(u)$ be the total number of instructions in one super iteration. Then, another lower bound for $t_{superiter}$ is

$$t_{lower} = \left\lceil \frac{\sum_{i=1}^{lcp} (I_{c_i}(u))}{p} \right\rceil + p_{stages} + n_{str}$$

Therefore,

$$t_{superiter} = \max\{t_{cp}, t_{lower}\}$$

The total loop execution time (t_{total}) is therefore:

number of iterations in the original loop = $|J|$
number of super iterations = (number of iterations in original loop) / (unrolling factor)
 $t_{total} = (\text{number of superiterations}) \cdot t_{superiter}$

Since the closing index increment and branch instruction of each super iteration was not considered for $t_{superiter}$ this will be taken into account for t_{total} . The time to execute an increment and branch instructions takes $2 \cdot (n_{str} + 1)$ cycles

$$t_{total} = \frac{|J|}{u} \cdot (t_{superiter} + 2 \cdot (n_{str} + 1))$$

V. THE OPTIMAL SOLUTION FOR THE LCS PROBLEM

In this section, the optimal unrolling factor for the LCS problem is derived. Then, the optimal execution time by loop unrolling along wavefront is compared with unrolling along the inner loop.

A. Finding the optimal unrolling factor along wavefronts

After the loop is unrolled along wavefronts, the assembly code and the DDG of each super iteration should be the one shown in figures 12 and 13. Respectively, except that u may not be two. In figure 13, there should be u sub data dependence graphs shown in figure 5. From figure 13, it is clear that $IC_1 = 2 \cdot u$, $IC_2 = u$, $IC_3 = 2 \cdot u$, $IC_4 = u$ and $IC_5 = u$. The optimal unrolling factor can be found by applying the first necessary condition for u_{opt} with the following assumptions: $p = 3$, $n_{str} = 2$, $p_{stage} = 5$. As shown before $lcp = 5$. By solving the equation this comes up with the solution $u_{opt} = 9$

The execution time of one super iteration can be calculated as follows.

$$\begin{aligned} \min[IC_1, IC_2, IC_3, IC_4, IC_5] &\geq p \cdot (n_{str} + 1) \\ u &\geq 3 \cdot (2 + 1) \\ u &\geq 9 \end{aligned}$$

$$\begin{aligned} t_{cp} &= lcp \cdot (n_{str} + 1) + p_{stage} = 5 \cdot 3 + 5 = 20 \\ t_{lower} &= \left\lceil \frac{\sum_{i=1}^{lcp} (I_{c_i}(u))}{p} \right\rceil + p_{stages} + n_{str} = 21 + 5 + 2 = 28 \\ t &= \max[t_{cp}, t_{lower}] = \max[20, 28] = 28 \end{aligned}$$

In order to demonstrate that this is the correct answer figure 17 shows the pipelines filled up with the calculated unrolling factor $u = 9$. The model shows 23 pipeline stages filled with instruction. With the additional 5 stages caused by the pipeline depth of p_{stage} stages the required time equals 28 cycles.

The total execution time of the nested loop is as seen before a function of the number of nodes in the iteration space J , the execution time for one super iteration $t_{superiter}$ and the unrolling factor u .

$$t_{total} = \frac{|J|}{u} \cdot (t_{superiter} + 2 \cdot (n_{str} + 1))$$

For the longest common subsequence problem this is equal to:

$$t_{total} = \frac{M \cdot N}{9} \cdot (28 + 6) = M \cdot N \cdot 3.77$$

Even though there was found a method to choose the optimal unrolling factor which saturates the pipelines with instructions and does not leave any stalls there may be stalls caused

	$\xleftrightarrow{n_{str}+1}$																						
P1	D1	D1	D1	D1	D1	D1	D2	D2	D2	D3	D3	D3	D3	D3	D3	D4	D4	D4	D5	D5	D5	x	x
P2	D1	D1	D1	D1	D1	D1	D2	D2	D2	D3	D3	D3	D3	D3	D3	D4	D4	D4	D5	D5	D5	x	x
P3	D1	D1	D1	D1	D1	D1	D2	D2	D2	D3	D3	D3	D3	D3	D3	D4	D4	D4	D5	D5	D5	x	x

Fig. 17. Filling the hardware's pipelines for the dependence graph in figure 13 with $u = 9$. The architecture's pipelines are supposed to be $p = 3$ and $n_{str} = 2$.

by wavefronts which number of nodes is exceeded by the unrolling factor.

B. Speed-up over unrolling along the inner loop

If we unroll the loop along the inner loop as shown in section III-A the critical path length grows linearly with the unrolling factor. Figure 18 shows the corresponding dependence graph. Since every node in this super iteration has a dependence to an instruction of the previous node the critical path length grows with each unrolling factor by one. For example the critical path length for $u = 2$ equals 6, whereas the critical path length for $u = 3$ equals 7. This has the effect, that the instruction counts $I_{c4}, I_{c5}, \dots$ are a constant and don't grow with the unrolling factor. In order to have a comparable result we use the same architectural parameters $p = 3$, $n_{str} = 2$ and $p_{stage} = 5$ as we did before. The first necessary condition

$$\min[I_{c1}, I_{c2}, \dots, I_{c_{lcp}}] \geq p \cdot (n_{str} + 1)$$

can not be satisfied in this case since there are constant instruction counts. Loop unrolling along the inner loop will not saturate the pipelines even with high unrolling factors.

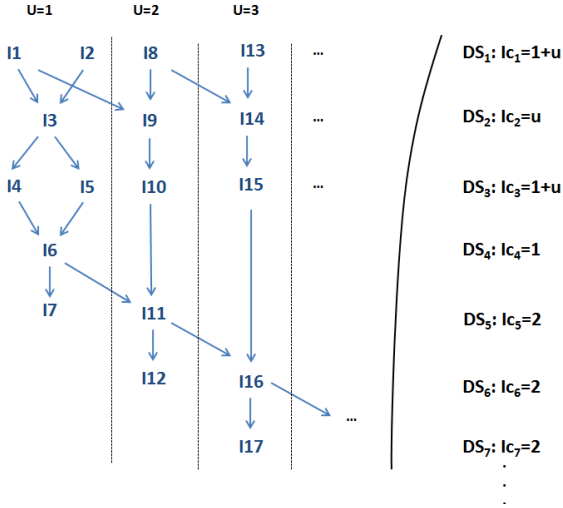


Fig. 18. Dependence graph for unrolling along the inner loop. The critical path length grows linear with the unrolling factor.

If we unroll along the inner loop of the original code with the same unrolling factor as we did by unrolling along the wavefronts ($u = 9$) it results in the following total execution time:

$$t_{totalold} = \frac{M \cdot N}{9} \cdot (39 + 6) = M \cdot N \cdot 5$$

It shows that the total number of instructions is at the technique of unrolling along the wavefronts with 47 instructions much less than with unrolling along the wavefronts (63 instructions for $u = 9$) due to the reuse of registers. However, this reuse of registers results also in a read-after-write dependence between all neighbour nodes which causes bubbles in the pipelines. Therefore the performance of the processor is not used optimally. The speed-up of unrolling along the wavefronts in comparison to the unrolling along the inner loop of the original loop is:

$$speedup = \frac{t_{totalold}}{t_{total}} = \frac{5}{3.77} = 1.326$$

VI. EXPERIMENTAL RESULTS

We benchmark the proposed technique on an Intel®Core™i5-2500 machine (3.3 GHz), a Sandy Bridge micro-architecture. This architecture supports out-of-order execution of instructions and consists of six pipelines where three pipelines perform arithmetic integer and floating point operations and three pipelines perform memory accesses. A more thorough description can be found in chapter 2.1 of [7].

A. The benchmark

We compare the longest common subsequence unrolled along wavefronts as implemented in figure 11 against the traditional method (figure 7). To eliminate the limitation that the dimension of the inner loop must be a multiple of the unrolling factor we apply *loop peeling*. Loop peeling is a method that moves the very first or the very last iterations out of the loop body [14]. Here we move the *DIMENI modulo u* last iterations of the inner loop to a separate loop, where *DIMENI* is the dimension of the inner loop. The problem is benchmarked with two randomly generated input strings of 25000 characters each, stored on the heap.

To measure the performance the *gettimeofday* linux system call is used [5]. This kernel routine takes use of the virtual dynamic shared object library (*VDSO*) of the linux kernel. Therefore, no switching from user space to kernel space is required, which minimizes the overhead of the measurement. The benchmarks are compiled using GCC 4.6.1 and tested on a linux platform with kernel version 2.6.35. The compilation process took advantage of all compiler optimizations by enabling option *-O3*. The optimizations performed do not conflict with our technique as they do not include loop unrolling. Each data point in figure 19 represents the arithmetic average of five repetitive runs.

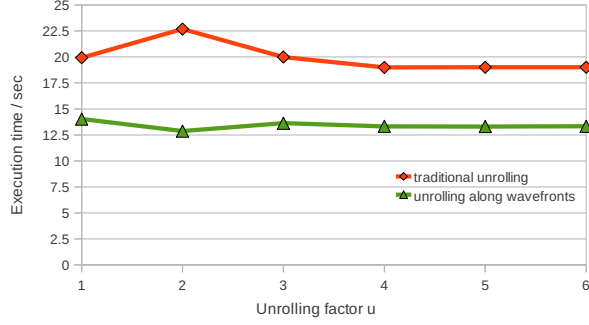


Fig. 19. The total execution time of the longest common subsequence problem on an Intel®Sandy Bridge micro-architecture.

B. Results and critical reflection

Figure 19 shows that the total execution time for increasing unrolling factors at each method. The proposed method executes consistently faster than the traditional implementation. Taking the lowest execution time of the traditional method 18.99sec at $u = 4$ in relation to the lowest execution time of the problem unrolled along wavefronts 12.87sec at $u = 2$ the speed-up achieved is 1.475. Of interest are especially the data points with unrolling factor $u = 1$, where no loop unrolling is applied. According to the theory presented in this paper the wavefront version of the loop nest should perform not as good as the traditional due to the additional loop overhead in form of min and max instructions. In fact, the transformed loop nest performs $19.93/14.03 = 1.42$ times better. As the architecture running the benchmark supports out-of-order execution, memory accesses can be pipelined if they are independent of each other. While the traditional implementation has loop carried dependencies in the inner loop the transformed code is free of loop carried dependencies along the inner loop and can take advantage of pipelined memory accesses.

The general shapes of the curves deviate from the shapes expected according to this paper, where the execution time should decrease with the unrolling factor until all pipelines are saturated. This is due to the abstract architecture model used throughout this paper which neglects memory and cache access latencies. The impact of memory and cache accesses on the unrolling direction and the optimal unrolling factor will be conducted in future research.

VII. CONCLUSION

This paper has demonstrated two major contributions for unrolling the LCS problem: unrolling a loop along wavefronts as well as two necessary conditions and a sufficient condition for the optimal unrolling factor. With unrolling along wavefronts, only independent iterations are grouped together and therefore the critical path length is independent of the unrolling factor. This means that it is possible to take the full advantage of the increase of parallelism due to higher unrolling factors and therefore reduce the total execution time. The objective is to saturate the pipelines with instructions such that the utilization of the processor hardware is maximized. The

two necessary conditions and the sufficient condition provide a guide of finding the optimal unrolling factors. Unrolling along the wavefronts applied on the LCS problem results in a speed-up of 1.475 in comparison to traditional unrolling on an Intel®Sandy Bridge micro-architecture.

As indicated by an example in this paper, the optimal unrolling factor may not exist if the critical path length increases faster than the number of instructions with the unrolling factor. One problem for future work is to find the optimal number of pipelines for this case. Furthermore, while this paper focused on exploiting instruction-level parallelism it ignored the impact of the register file's size as well as the importance of data locality. The combination of those three aspects will be conducted in future research. Also, the performance of this technique may be further increased by applying software pipelining.

REFERENCES

- [1] M. M. Baskaran, A. Hartono, S. Tavarageri, T. Henretty, J. Ramanujam, and P. Sadayappan. Parameterized tiling revisited. In *Proceedings of the 8th annual IEEE/ACM international symposium on Code generation and optimization*, CGO '10, pages 200–209, New York, NY, USA, 2010. ACM.
- [2] C. Bastoul. Code generation in the polyhedral model is easier than you think. In *PACT'13*, pages 7–16, Juan-les-Pins, France, September 2004.
- [3] B. Ben Mabrouk, H. Hasni, and Z. Mahjoub. Parallelization of the dynamic programming algorithm for solving the longest common subsequence problem. In *Computer Systems and Applications (AICCSA), 2010 IEEE/ACS International Conference on*, pages 1–8, may 2010.
- [4] U. Bondhugula, J. Ramanujam, and P. Sadayappan. Pluto: A practical and fully automatic polyhedral parallelizer and locality optimizer. Technical Report OSU-CISRC-10/07-TR70, The Ohio State University, Oct. 2007.
- [5] D. Bovet and M. Cesati. *Understanding The Linux Kernel*. Oreilly & Associates Inc, 2005.
- [6] T. Cormen, C. Leiserson, R. Rivest, and C. Stein. *Introduction to algorithms*. MIT Press, Cambridge, MA, USA, 2001.
- [7] Intel. Intel®64 and ia-32 architectures optimization reference manual. order number: 248966-025. June 2011.
- [8] L. Lamport. The parallel execution of do loops. *Commun. ACM*, 17:83–93, February 1974.
- [9] C. Lang. (r) polynomial - time nested loop fusion with full parallelism. In *ICPP '96*, page 9, Washington, DC, USA, 1996. IEEE Computer Society.
- [10] N. Manjikian and T. Abdelrahman. Scheduling of wavefront parallelism on scalable shared-memory multiprocessors. In *Parallel Processing, 1996., Proceedings of the 1996 International Conference on*, volume 3, pages 122–131 vol.3, aug 1996.
- [11] W. S. and J. A. B. Fortes. Time optimal linear schedules for algorithms with uniform dependencies. *IEEE Trans. Comput.*, 40(6):723–742, 1991.
- [12] V. Sarkar. Optimized unrolling of nested loops. *Int. J. Parallel Program.*, 29:545–581, October 2001.
- [13] J. Shin, C. Jacqueline, and M. W. Hall. Compiler-controlled caching in superword register files for multimedia extension architectures. In *Proceedings of the 2002 International Conference on Parallel Architectures and Compilation Techniques*, PACT '02, pages 45–55, Washington, DC, USA, 2002. IEEE Computer Society.
- [14] L. Song and K. Kavi. A technique for variable dependence driven loop peeling. In *Algorithms and Architectures for Parallel Processing, 2002. Proceedings. Fifth International Conference on*, pages 390–395, 2002.
- [15] M. Stephenson and S. Amarasinghe. Predicting unroll factors using supervised classification. In *Proceedings of the international symposium on Code generation and optimization*, CGO '05, pages 123–134, Washington, DC, USA, 2005. IEEE Computer Society.
- [16] M. E. Wolf and M. S. Lam. A loop transformation theory and an algorithm to maximize parallelism. *IEEE Trans. Parallel Distrib. Syst.*, 2(4):452–471, 1991.
- [17] M. E. Wolf, D. E. Maydan, and D.-K. Chen. Combining loop transformations considering caches and scheduling. In *MICRO 29*, pages 274–286, Washington, DC, USA, 1996. IEEE Computer Society.